

# CSE341 – Microprocessors

## Lab Final Exam – Scenario-Based Practice Problems

*Covering: Lab 01 → Lab 02 → Lab 03A → Lab 03B (Lab 04)*

*Scenario • Code with Comments • Viva Q&A*



## Quick Concept Map — What's on the Final

| Lab     | Key Concepts                                                                                                                         |
|---------|--------------------------------------------------------------------------------------------------------------------------------------|
| Lab 01  | Registers (AX/BX/CX/DX + AH/AL etc.), MOV, ADD, SUB, MUL, DIV, INC, DEC, NEG, DB/DW variables, hex/binary/decimal notation           |
| Lab 02  | INT 21H (AH=1 input, AH=2 output, AH=9 string), LEA, ASCII conversion (SUB/ADD 30h), upper↔lower case (±20h), flags (CF,ZF,SF,OF,PF) |
| Lab 03A | JMP, CMP, conditional jumps (JE,JNE,JG,JL,JGE,JLE), IF-THEN, IF-THEN-ELSE, CASE/multiway branch                                      |
| Lab 03B | Explicit loop (WHILE: CMP+JMP), Implicit loop (FOR: LOOP+CX), Repeat-Until (DO-WHILE: body first + JNE/JE at end), nested loops      |



### THINGS EXAMINERS ALWAYS ASK:

- Why do we write MOV AX,@DATA then MOV DS,AX? (can't load DS directly from constant)
- Why save AL to BL immediately after INT 21H input? (INT 21H destroys AX)
- What is the difference between WHILE and REPEAT-UNTIL? (condition check position)
- Why is LOOP special? (always uses CX — never use CX for other data inside LOOP)
- What does CMP do? (subtracts internally, sets flags, does NOT change either register)
- In IF-THEN, why is the jump inverted? (we jump OVER the body when condition is FALSE)
- MUL result location: byte→AX, word→DX:AX. DIV result: byte→quotient AL, remainder AH



## Problem 1: The ATM PIN Validator



TOPICS COVERED: Lab 01: MOV/Arithmetic • Lab 02: I/O + ASCII • Lab 03A: IF-THEN-ELSE branching

### Scenario:

You are building a simple ATM system for a local bank. The ATM has a 2-digit PIN. The correct PIN is hardcoded as 42 (for simplicity). The ATM must:

1. Display: 'ENTER PIN: '
2. Accept a 2-digit number from the user (two separate digit inputs)
3. Reconstruct the full 2-digit number from the two digits
4. Compare with the correct PIN (42)
5. If correct → display 'ACCESS GRANTED'
6. If wrong → display 'ACCESS DENIED'

The bank manager also wants to know: what if someone enters a number larger than the PIN? What if smaller? The system must handle all cases.



### Your Task:

- Read two digit characters one by one
- Reconstruct the 2-digit number:  $\text{num} = (\text{tens} * 10) + \text{units}$
- Compare with 42 using CMP and conditional branch (IF-THEN-ELSE)
- Display appropriate message based on comparison result
- Use .MODEL SMALL template with .DATA and .CODE sections

### Complete Solution (with line-by-line comments):

```
.MODEL SMALL
.STACK 100H
.DATA
    prompt DB "ENTER PIN: $"
    granted DB "ACCESS GRANTED$"
    denied DB "ACCESS DENIED$"
    newl DB 13, 10, "$" ; CR+LF for new line

.CODE
MAIN PROC
    MOV AX, @DATA ; load address of data segment into AX
    MOV DS, AX ; set DS - can't load DS directly from constant

    ; === Display prompt ===
    LEA DX, prompt ; LEA loads ADDRESS of 'prompt' into DX, not its value
    MOV AH, 9 ; AH=9 tells INT 21H: print string at address in DX
    INT 21H ; DOS prints every byte from prompt until it hits '$'
```

```

; === Read tens digit ===
MOV AH, 1          ; AH=1 = keyboard input service
INT 21H           ; waits for keypress; ASCII value → AL
SUB AL, 30h        ; convert ASCII to numeric: '4'(52) - 48 = 4
MOV BL, AL         ; save tens digit in BL — INT 21H will
destroy AL next call

; === Read units digit ===
MOV AH, 1
INT 21H           ; ASCII of second digit → AL
SUB AL, 30h        ; convert: '2'(50) - 48 = 2
MOV CL, AL         ; save units digit in CL

; === Reconstruct full number: (tens * 10) + units ===
MOV AL, BL         ; AL = tens digit (4)
MOV AH, 0          ; clear AH so AX = just AL (avoid garbage in
high byte)
MOV DL, 10         ; DL = multiplier
MUL DL             ; AX = AL * DL = 4 * 10 = 40 (byte mul:
result in AX)
MOV AH, 0          ; clear AH again after MUL (MUL can set AH)
ADD AL, CL         ; AL = 40 + 2 = 42 (the full PIN entered)

; === Compare entered PIN with correct PIN (42) ===
CMP AL, 42         ; AL - 42 internally; sets flags; does NOT
change AL
JNE access_denied  ; if AL != 42 → jump to denied message

; === Correct PIN: IF branch ===
LEA DX, newl       ; print new line first
MOV AH, 9
INT 21H
LEA DX, granted    ; load address of 'ACCESS GRANTED' string
MOV AH, 9
INT 21H            ; print it
JMP done           ; MUST jump over denied section — without
this both print!

access_denied:      ; ELSE branch — only reaches here if PIN was
wrong
    LEA DX, newl
    MOV AH, 9
    INT 21H
    LEA DX, denied  ; load address of 'ACCESS DENIED' string
    MOV AH, 9
    INT 21H

done:              ; both branches meet here — program exits
    MOV AX, 4C00H    ; AH=4Ch = terminate program, AL=00 = exit
code 0
    INT 21H         ; tell DOS: we are done, return control to OS
MAIN ENDP
END MAIN

```



## Viva Preparation Q&A:

| #   | Viva Question                                                   | Model Answer                                                                                                                                                                                                                                              |
|-----|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q 1 | Why MOV AX,@DATA then MOV DS,AX? Why not MOV DS,@DATA directly? | Hardware restriction: segment registers (DS,CS,SS,ES) cannot receive immediate/constant values directly. @DATA is a constant (the segment address). It must first go into a general register (AX), then be copied to DS. This is a fundamental 8086 rule. |
| Q 2 | Why SUB AL, 30h after reading input?                            | INT 21H stores the ASCII value in AL, not the numeric digit. '4' has ASCII value 52. To get the actual number 4, subtract 48 (30h): 52-48=4. Without this, you'd be comparing ASCII values, not actual numbers.                                           |
| Q 3 | Why MOV AH,0 before MUL?                                        | Byte MUL multiplies AL * source → result in AX. AH is the high byte of AX. If AH has leftover garbage (e.g., 5), AX=0504h before MUL, giving completely wrong result. MOV AH,0 ensures AX = just the digit value in AL.                                   |
| Q 4 | Why is JMP done needed after the granted section?               | Without JMP done, after printing ACCESS GRANTED, execution falls straight into access_denied and prints ACCESS DENIED too. JMP done forces a skip over the else-block — this is mandatory in every IF-THEN-ELSE structure.                                |
| Q 5 | What does CMP AL,42 actually do?                                | CMP computes AL-42 internally and sets flags (ZF if equal, SF if negative etc.) but does NOT store the result and does NOT change AL. It only updates the flag register. The following JNE then reads those flags to decide whether to jump.              |
| Q 6 | What is JNE and when does it jump?                              | JNE = Jump if Not Equal. It jumps when ZF=0 (Zero Flag is 0), meaning the previous CMP result was NOT zero (i.e., operands were not equal). Here: if AL != 42, ZF=0, JNE jumps to access_denied.                                                          |



## Problem 2: The Vending Machine Grade Classifier



TOPICS COVERED: Lab 02: I/O + ASCII • Lab 03A: CASE structure • Lab 01: Arithmetic

### Scenario:

You are programming the display unit of a student grade vending machine at BRACU. When a student inserts their score (a single digit 1-9 representing grade points), the machine must display their letter grade:

Score 9 → display 'A+'  
Score 8 → display 'A'  
Score 7 → display 'B'  
Score 4,5,6 → display 'C'  
Score 1,2,3 → display 'D'  
Anything else → display 'INVALID'

This is a CASE structure problem — multiple distinct values mapping to different outputs. After displaying the grade, also display: 'Have a good day!' on the next line.



### Your Task:

- Read a single character input (the grade point)
- Use CASE structure: multiple CMP+JE chains to check each possible value
- For ranges (4-6, 1-3), use range comparison (CMP + JGE/JLE)
- Each case prints its grade letter then jumps to end (like break in switch)
- Print 'Have a good day!' after regardless of grade

### Complete Solution (with line-by-line comments):

```
.MODEL SMALL
.STACK 100H
.DATA
    ask      DB "ENTER GRADE POINT (1-9): $"
    gradeAP  DB "A+$"
    gradeA   DB "A$"
    gradeB   DB "B$"
    gradeC   DB "C$"
    gradeD   DB "D$"
    invalid  DB "INVALID$"
    goodbye  DB "Have a good day!$"
    newl     DB 13, 10, "$"

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, ask          ; prompt the student
    MOV AH, 9
```

```

INT 21H

MOV AH, 1          ; read single character input
INT 21H            ; ASCII of digit → AL (e.g., '7' = ASCII 55)
MOV BL, AL         ; save immediately – next INT 21H destroys AL

LEA DX, newl       ; move to next line before printing grade
MOV AH, 9
INT 21H

; === CASE structure: check BL against each grade value ===
; Note: BL holds ASCII character, so compare with ASCII literals

CMP BL, '9'        ; is it '9' (ASCII 57)?
JE  case_AP        ; yes → A+

CMP BL, '8'        ; is it '8'?
JE  case_A

CMP BL, '7'        ; is it '7'?
JE  case_B

; === Range check for C: 4,5,6 ===
CMP BL, '4'        ; is BL >= '4'?
JL  check_D        ; if BL < '4', skip this range – not 4,5,6
CMP BL, '6'        ; is BL <= '6'?
JLE case_C        ; if yes (4 <= BL <= 6) → grade C

check_D:
; === Range check for D: 1,2,3 ===
CMP BL, '1'        ; is BL >= '1'?
JL  case_invalid   ; if BL < '1' (e.g., '0') → invalid
CMP BL, '3'        ; is BL <= '3'?
JLE case_D         ; if yes (1 <= BL <= 3) → grade D
JMP case_invalid   ; none matched → invalid

case_AP:
LEA DX, gradeAP    ; load address of 'A+' string
MOV AH, 9
INT 21H
JMP print_goodbye  ; CRITICAL: jump to end – prevents
fall-through to A case

case_A:
LEA DX, gradeA
MOV AH, 9
INT 21H
JMP print_goodbye  ; each case needs its own JMP – like 'break'
in switch

case_B:
LEA DX, gradeB
MOV AH, 9

```

```

    INT 21H
    JMP print_goodbye

case_C:
    LEA DX, gradeC
    MOV AH, 9
    INT 21H
    JMP print_goodbye

case_D:
    LEA DX, gradeD
    MOV AH, 9
    INT 21H
    JMP print_goodbye

case_invalid:
    LEA DX, invalid
    MOV AH, 9
    INT 21H

print_goodbye:                ; ALL cases land here — common exit point
    LEA DX, newl
    MOV AH, 9
    INT 21H
    LEA DX, goodbye
    MOV AH, 9
    INT 21H

    MOV AX, 4C00H
    INT 21H
MAIN ENDP
END MAIN

```

### Viva Preparation Q&A:

| #          | Viva Question                                                                    | Model Answer                                                                                                                                                                                                                                                                        |
|------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Q 1</b> | <b>What is the CASE structure in assembly? How is it different from IF-ELSE?</b> | CASE uses a chain of CMP+JE instructions, one per value. IF-ELSE has one condition with two paths. CASE has N conditions with N paths. Each case block ends with JMP end_case (like break). IF-ELSE needs only 1 conditional jump + 1 JMP. CASE needs N conditional jumps + N JMPs. |
| <b>Q 2</b> | <b>Why compare BL with '4' not just 4 in the range check?</b>                    | BL holds the ASCII value of the typed character. '4' written in single quotes = ASCII 52. If you compare with numeric 4, you're comparing 52 with 4 — always false! When using character input without conversion, compare with character literals or add 30h to the numeric value. |
| <b>Q 3</b> | <b>How does range checking work? Why two CMP instructions for one range?</b>     | A range [4,6] needs two boundaries checked: (1) CMP BL,'4'/JL exit — if below lower bound, not in range. (2) CMP BL,'6'/JLE case_C — if at or below upper bound AND passed lower check, it's in range. One CMP can only check one boundary at a time.                               |

|                |                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Q<br/>4</b> | <b>What happens if you forget <code>JMP print_goodbye</code> after <code>case_AP</code>?</b> | Execution falls through into <code>case_A</code> and prints 'A' immediately after 'A+'. Then falls into <code>case_B</code> , prints 'B', falls into <code>case_C</code> ... prints everything. In assembly there's NO automatic break — you must explicitly <code>JMP</code> to exit each case.                                                                                          |
| <b>Q<br/>5</b> | <b>Why is <code>BL</code> used instead of keeping the input in <code>AL</code>?</b>          | <code>INT 21H</code> ( <code>AH=9</code> ) for string printing destroys <code>AX</code> . When we call <code>INT 21H</code> to print the newline string, <code>AL</code> gets overwritten. If the input was still in <code>AL</code> , it would be lost. <code>BL</code> is not touched by <code>INT 21H</code> , so the input character stays safe across all the string printing calls. |
| <b>Q<br/>6</b> | <b>In what order should you place <code>CASE</code> checks for best efficiency?</b>          | Place the most commonly entered values first (they match earlier and exit faster). Also place individual value checks ( <code>JE</code> ) before range checks — ranges require 2 <code>CMPs</code> vs 1 for exact match. Order matters for performance but not correctness (all valid inputs still reach their case).                                                                     |



## Problem 3: The Smart Cafeteria Bill Counter



TOPICS COVERED: Lab 01: MUL/DIV • Lab 02: I/O + ASCII • Lab 03B: LOOP (implicit) • Lab 03A: Branching

### Scenario:

The BRACU cafeteria has a new smart billing system. Students can buy items repeatedly. Each item costs a fixed price per unit. The cashier enters the quantity (1-9 units) and the price per unit (1-9 Taka) for each item. The system repeats this for exactly 3 items, calculates the total bill, and then:

- If total bill > 50 Taka → print 'DISCOUNT APPLIED' (you get 10 Taka off) and print the discounted total
- If total ≤ 50 Taka → print 'NO DISCOUNT' and print the total as-is

The cashier enters qty and price as single digits. You must use a LOOP for the 3-item iteration, and IF-ELSE for the discount check.



### Your Task:

- Use implicit LOOP with CX=3 for 3 items
- Inside loop: read qty (digit), read price (digit), multiply → add to running total
- After loop: CMP total with 50 → branch for discount
- Display final amount using AAM for 2-digit output
- Combine MUL, ADD, LOOP, CMP, JG, INT 21H all in one program

### Complete Solution (with line-by-line comments):

```

.MODEL SMALL
.STACK 100H
.DATA
    msgQty    DB "Enter qty    (1-9): $"
    msgPrice  DB "Enter price (1-9): $"
    msgDisc   DB "DISCOUNT APPLIED! Total: $"
    msgNone   DB "NO DISCOUNT. Total: $"
    msgTaka   DB " Taka$"
    newl      DB 13, 10, "$"

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, 3                ; CX = loop counter = 3 items (LOOP
instruction uses CX!)
    MOV DH, 0                ; DH = running total bill (starts at 0)
                                ; DH chosen because LOOP uses CX, MUL uses AX
- DH is safe

```

```

ITEM_LOOP:
    ; --- Print newline for clean display ---
    LEA DX, newl
    MOV AH, 9
    INT 21H
    ; NOTE: LEA DX,newl overwrites DX - that's fine, we reload it every
time

    ; --- Read quantity ---
    LEA DX, msgQty
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H                ; qty character → AL
    SUB AL, 30h            ; ASCII '3' (51) → numeric 3
    MOV BL, AL             ; BL = qty - safe from next INT 21H call

    ; --- Read price ---
    LEA DX, msgPrice
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H                ; price character → AL
    SUB AL, 30h            ; ASCII → numeric
    MOV CL, AL             ; CL = price
                            ; WARNING: CX is our LOOP counter!
                            ; CL = low byte of CX. After MUL we restore CX
properly.
                            ; Since LOOP only checks CX at LOOP
instruction (end),
                            ; using CL here is safe IF we restore CX
before LOOP executes.
                            ; Actually here CL overwrite is a risk - use
BH instead:

    ; SAFER: use BH for price instead of CL
    MOV BH, AL             ; BH = price per unit (use BH not CL to
protect CX!)

    ; --- Calculate item cost: qty * price ---
    MOV AL, BL             ; AL = qty (MUL byte: one operand MUST be in
AL)
    MOV AH, 0              ; clear AH so AX = qty only
    MUL BH                 ; AX = AL * BH = qty * price
                            ; result in AX (byte mul of AL * BH → 16-bit
in AX)

    ; --- Add item cost to running total ---
    ADD DH, AL             ; DH = DH + item_cost (assuming total fits in
8 bits: max 81*3=243, fits)
                            ; DH is our accumulator - safe from MUL, INT
21H, LOOP

    LOOP ITEM_LOOP         ; CX-- and jump back if CX != 0

```

```

                                ; after 3rd iteration CX becomes 0 → loop
exits

    ; === Discount check after loop ===
    CMP DH, 50                  ; compare total with 50
    JLE no_discount            ; if total <= 50 → no discount

    ; === Total > 50: apply 10 Taka discount ===
    SUB DH, 10                  ; DH = total - 10 (discount applied)
    LEA DX, new1
    MOV AH, 9
    INT 21H
    LEA DX, msgDisc             ; print 'DISCOUNT APPLIED! Total: '
    MOV AH, 9
    INT 21H
    JMP print_total            ; jump to display total

no_discount:
    LEA DX, new1
    MOV AH, 9
    INT 21H
    LEA DX, msgNone             ; print 'NO DISCOUNT. Total: '
    MOV AH, 9
    INT 21H

print_total:
    ; Display DH as 2-digit number using AAM
    MOV AL, DH                  ; AL = total value
    AAM                        ; AH = tens digit, AL = units digit (AL / 10)
                                ; e.g. total=63: AH=6, AL=3
    ADD AH, 30h                 ; tens → ASCII
    ADD AL, 30h                 ; units → ASCII
    MOV BH, AH                  ; save tens ASCII (INT 21H will destroy AH)
    MOV BL, AL                  ; save units ASCII

    MOV DL, BH                  ; print tens digit
    MOV AH, 2
    INT 21H

    MOV DL, BL                  ; print units digit
    MOV AH, 2
    INT 21H

    LEA DX, msgTaka             ; print ' Taka'
    MOV AH, 9
    INT 21H

    MOV AX, 4C00H
    INT 21H
MAIN ENDP
END MAIN

```

## Viva Preparation Q&A:

| #   | Viva Question                                                                  | Model Answer                                                                                                                                                                                                                                                                                         |
|-----|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q 1 | Why must LOOP always use CX? What happens if CX is accidentally used for data? | LOOP is hardwired to CX — it always decrements CX and checks CX==0. If you store data in CX (or CL/CH) inside the loop body, LOOP reads a wrong counter and either loops wrong number of times or runs forever. Always protect CX when using LOOP: use DH, BH, SI, DI etc. for data inside the loop. |
| Q 2 | Why is DH used as the running total instead of a memory variable?              | DH (high byte of DX) is not used by LOOP, MUL, or INT 21H in this code. It persists across all loop iterations and INT 21H calls. Memory variables also work but require MOV instructions to read/write. Registers are faster. Any 'safe' register works — DH just happens to be free here.          |
| Q 3 | Why does MUL BH give the right result here?                                    | Byte MUL: $AX = AL * \text{source (8-bit)}$ . $AL = \text{qty}$ , $BH = \text{price}$ . MUL BH means $AX = AL * BH = \text{qty} * \text{price}$ . Result in AX (16-bit). We only use AL since $\text{max}=81$ fits in 8 bits. MUL always uses AL implicitly for one operand.                         |
| Q 4 | What does AAM do and why is it used here?                                      | AAM (ASCII Adjust for Multiply) divides AL by 10: $AH = \text{quotient (tens)}$ , $AL = \text{remainder (units)}$ . e.g. $AL=63 \rightarrow AH=6, AL=3$ . Adding 30h to each converts to ASCII '6','3'. Used to display any 2-digit result (0-99) as two separate printable characters.              |
| Q 5 | What is the maximum possible total and can DH hold it?                         | Max qty=9, max price=9, max item cost=81. 3 items: $81*3=243$ . DH is 8-bit (max 255). $243 < 255$ so DH can hold it. If items could exceed 255, we'd need DX (16-bit) as accumulator.                                                                                                               |
| Q 6 | Why does JLE no_discount cover both less-than AND equal-to cases?              | JLE = Jump if Less or Equal ( $ZF=1$ OR $SF=OF$ ). After $CMP\ DH, 50$ : if $DH < 50$ OR $DH = 50 \rightarrow$ jump to no_discount. Exactly the logic we want: discount only when total STRICTLY $> 50$ . JL would miss the equal case (50 Taka = no discount, which is correct behavior).           |



## Problem 4: The Security Keypad — Password Retry System



TOPICS COVERED: Lab 02: I/O • Lab 03A: IF-THEN • Lab 03B: Repeat-Until + Explicit Loop

### Scenario:

You are programming a security door keypad for a server room. The door opens only when the correct 3-character password is entered. The password is: 'CSE'

The system works as follows:

1. Display: 'ENTER 3-CHAR PASSWORD: '
2. Read 3 characters from user one by one (display as typed)
3. Check all 3 characters against 'C', 'S', 'E' respectively
4. If ALL 3 match → print 'DOOR OPEN' and exit
5. If ANY mismatch → print 'WRONG PASSWORD. TRY AGAIN.' and repeat from step 1
6. The system keeps asking until the correct password is entered (infinite retry)

This tests your ability to combine Repeat-Until (outer retry loop) with sequential IF-THEN checks (inner character matching).



### Your Task:

- Outer loop: REPEAT-UNTIL style — keep asking until password correct
- Inner: read 3 chars using LOOP (CX=3) or sequential reads
- Check each char: CMP + JNE (branch to wrong\_pass if any mismatch)
- If all 3 match: fall through to DOOR OPEN and exit
- If mismatch: print error, jump back to top of outer loop

### Complete Solution (with line-by-line comments):

```

.MODEL SMALL
.STACK 100H
.DATA
    ask    DB "ENTER 3-CHAR PASSWORD: $"
    open   DB "DOOR OPEN$"
    wrong  DB "WRONG PASSWORD. TRY AGAIN.$"
    newl   DB 13, 10, "$"

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; =====
    ; OUTER REPEAT-UNTIL loop: retry until correct
    ; =====

RETRY:                                ; label at TOP — but condition checked at
BOTTOM

```

```

; This is REPEAT-UNTIL: body runs first,
check after
    LEA DX, newl
    MOV AH, 9
    INT 21H

    LEA DX, ask           ; print 'ENTER 3-CHAR PASSWORD: '
    MOV AH, 9
    INT 21H

; === Read 1st character ===
    MOV AH, 1
    INT 21H               ; 1st char → AL
    MOV BL, AL            ; save 1st char in BL – will survive next
INT 21H

; === Read 2nd character ===
    MOV AH, 1
    INT 21H               ; 2nd char → AL
    MOV CL, AL            ; save 2nd char in CL

; === Read 3rd character ===
    MOV AH, 1
    INT 21H               ; 3rd char → AL
    MOV BH, AL            ; save 3rd char in BH

; =====
; Check all 3 characters against 'C','S','E'
; =====

    CMP BL, 'C'           ; is 1st char 'C'?
    JNE wrong_pass        ; if NOT 'C' → wrong → jump to error
message
; JNE checks ZF: if ZF=0 (not equal) →
jump

    CMP CL, 'S'           ; is 2nd char 'S'?
    JNE wrong_pass        ; if NOT 'S' → wrong

    CMP BH, 'E'           ; is 3rd char 'E'?
    JNE wrong_pass        ; if NOT 'E' → wrong

; === All 3 matched! ===
    LEA DX, newl
    MOV AH, 9
    INT 21H
    LEA DX, open          ; print 'DOOR OPEN'
    MOV AH, 9
    INT 21H
    JMP exit_program      ; exit – door is open, no more retries
needed

wrong_pass:              ; any character mismatch lands here
    LEA DX, newl

```

```

MOV AH, 9
INT 21H
LEA DX, wrong           ; print 'WRONG PASSWORD. TRY AGAIN.'
MOV AH, 9
INT 21H
JMP RETRY               ; jump back to TOP of outer loop — try
again
                        ; This is the 'UNTIL correct' part of
REPEAT-UNTIL
                        ; We loop back unconditionally and
re-check at bottom

exit_program:
MOV AX, 4C00H
INT 21H
MAIN ENDP
END MAIN

```



### Viva Preparation Q&A:

| #   | Viva Question                                                        | Model Answer                                                                                                                                                                                                                                                                                                                                                    |
|-----|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q 1 | This uses REPEAT-UNTIL style. How is it different from a WHILE loop? | WHILE checks condition BEFORE body — may never execute. REPEAT-UNTIL executes body FIRST, checks at the END. Here the prompt always appears before any check — that's repeat-until. Also: WHILE needs 2 jumps (conditional at top + JMP at bottom). REPEAT-UNTIL needs only 1 JMP at bottom (JMP RETRY) and 1 conditional exit (JMP exit_program when correct). |
| Q 2 | Why are BL, CL, BH chosen for the 3 characters?                      | INT 21H destroys AX (including AL and AH). Each input call wipes out the previous AL. BL, CL, BH are not touched by INT 21H. BH (high byte of BX) is a safe 8-bit register often overlooked. Together BL+CL+BH can hold 3 characters across multiple INT 21H calls.                                                                                             |
| Q 3 | What is the logic flow when the user types 'CSX' (wrong 3rd char)?   | BL='C', CL='S', BH='X'. CMP BL,'C' → equal → JNE doesn't jump. CMP CL,'S' → equal → JNE doesn't jump. CMP BH,'E' → NOT equal ('X'≠'E') → ZF=0 → JNE jumps to wrong_pass. Prints error, JMP RETRY, loops back. Tries again from RETRY label.                                                                                                                     |
| Q 4 | Is this loop guaranteed to terminate?                                | No! If the user never enters 'CSE', the loop runs forever. This is intentional — security systems keep asking until correct password is entered. In a real system, you'd add a maximum attempt counter (e.g., lock after 3 wrong tries) using a WHILE loop with counter.                                                                                        |
| Q 5 | What if the user types lowercase 'cse' instead of 'CSE'?             | 'c'=99, 'C'=67. They differ by 32 (20h). CMP BL,'C' would fail since BL='c'(99)≠'C'(67). To accept both: ADD BL,0 → check CMP BL,'c' as additional option with JE, OR: if BL>='a' AND BL<='z', SUB BL,20h to normalize to uppercase before comparing.                                                                                                           |
| Q 6 | Why does wrong_pass use JMP RETRY instead of a conditional jump?     | By the time we reach wrong_pass, we already know the password is wrong (we only get there via JNE). So the jump back to RETRY is unconditional — no need                                                                                                                                                                                                        |

to check anything. JMP (unconditional) is used when the decision is already made. Conditional jumps are only needed when we have two possible paths.



## Problem 5: The Student Marks Analyzer — Running Average



TOPICS COVERED: Lab 01: ADD/DIV • Lab 02: I/O • Lab 03B: Explicit WHILE loop • Lab 03A: IF-THEN • Lab 02: Flags

### Scenario:

The CSE department needs a marks analyzer for their quiz system. A TA enters student marks one by one (single digits 1-9). The system keeps accepting marks until the TA enters '0' as a stop signal.

After all marks are entered, the system must:

1. Display total count of students (how many marks were entered)
2. Display the sum of all marks
3. Calculate and display the average (sum / count, integer division)
4. If average  $\geq 7 \rightarrow$  display 'CLASS PERFORMANCE: GOOD'  
If average  $< 7 \rightarrow$  display 'CLASS PERFORMANCE: NEEDS IMPROVEMENT'

The TA may enter up to 9 students. This is a WHILE loop (condition checked first) because if TA immediately enters '0', nothing should be processed.

Marks: 8, 6, 7, 5, 0  $\rightarrow$  count=4, sum=26, avg=6, output: NEEDS IMPROVEMENT



### Your Task:

- $\rightarrow$  WHILE loop (explicit): check for '0' BEFORE processing each mark
- $\rightarrow$  Accumulate sum in BX, count in DH
- $\rightarrow$  Stop when '0' is entered (Repeat-Until alternative would process the 0 — wrong!)
- $\rightarrow$  After loop: DIV to get average, then CMP for performance message
- $\rightarrow$  Combines WHILE, I/O, arithmetic, branching all together

### Complete Solution (with line-by-line comments):

```

.MODEL SMALL
.STACK 100H
.DATA
    ask      DB "Enter mark (0 to stop): $"
    msgCnt   DB "Count: $"
    msgSum   DB "Sum: $"
    msgAvg   DB "Average: $"
    msgGood  DB "CLASS PERFORMANCE: GOOD$"
    msgBad   DB "CLASS PERFORMANCE: NEEDS IMPROVEMENT$"
    newl     DB 13, 10, "$"

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV BX, 0                ; BX = sum accumulator (16-bit for safety)
    MOV DH, 0                ; DH = count of students entered

```

```

; =====
; WHILE loop: check stop condition FIRST
; If TA enters '0' immediately, loop body never runs
; =====
WHILE TOP:
    LEA DX, new1
    MOV AH, 9
    INT 21H

    LEA DX, ask          ; prompt TA for next mark
    MOV AH, 9
    INT 21H

    MOV AH, 1            ; read one character
    INT 21H              ; ASCII of digit → AL
    MOV CL, AL           ; save entered character before INT 21H
    destroys AL

    ; === Check stop condition BEFORE processing ===
    CMP CL, '0'          ; did TA enter '0'?
    JE WHILE_END         ; YES → exit loop immediately (WHILE style:
check first!)

                                ; This is what makes it WHILE not REPEAT-UNTIL
                                ; If '0' entered on first try, BX and DH stay
0

    ; === Process the mark (only if NOT '0') ===
    SUB CL, 30h          ; convert ASCII to numeric: '8'(56) → 8
    MOV AL, CL
    MOV AH, 0            ; clear AH so AX = just the mark value
    ADD BX, AX            ; BX = BX + mark (accumulate sum)
    INC DH              ; DH = DH + 1 (count one more student)

    JMP WHILE_TOP        ; jump back to TOP to check next input
                                ; This unconditional JMP is the 'back edge' of
WHILE loop

WHILE_END:              ; loop exits here when '0' is entered

    ; === Display count ===
    LEA DX, new1
    MOV AH, 9
    INT 21H
    LEA DX, msgCnt
    MOV AH, 9
    INT 21H
    MOV DL, DH           ; count is in DH
    ADD DL, 30h          ; convert to ASCII (count 1-9, single digit)
    MOV AH, 2
    INT 21H

    ; === Display sum (may be 2 digits: max 9*9=81) ===

```

```

LEA DX, new1
MOV AH, 9
INT 21H
LEA DX, msgSum
MOV AH, 9
INT 21H
MOV AL, BL          ; sum low byte into AL for AAM
AAM                 ; AH=tens, AL=units
ADD AH, 30h         ; tens → ASCII
ADD AL, 30h         ; units → ASCII
MOV CH, AH          ; save tens (INT 21H will destroy AH!)
MOV CL, AL          ; save units
MOV DL, CH
MOV AH, 2
INT 21H             ; print tens
MOV DL, CL
MOV AH, 2
INT 21H             ; print units

; === Calculate average = sum / count ===
MOV AL, BL          ; AL = sum (dividend must be in AL for byte
div)
MOV AH, 0           ; AH=0 so AX = sum (not garbage)
MOV BL, DH          ; BL = count (divisor – must be register, not
constant)
DIV BL              ; AL = quotient (average), AH = remainder
(ignored)
MOV SI, AX          ; save quotient in SI before INT 21H destroys
AX

; === Display average ===
LEA DX, new1
MOV AH, 9
INT 21H
LEA DX, msgAvg
MOV AH, 9
INT 21H
MOV AL, SIL         ; restore average into AL (SI low byte)
; Simple: average is 1-9, single digit
ADD AL, 30h         ; numeric → ASCII
MOV DL, AL
MOV AH, 2
INT 21H

; === Performance check: IF average >= 7 THEN good ELSE needs
improvement ===
MOV AL, SIL         ; reload average
CMP AL, 7           ; compare average with 7
JL needs_improvement ; if average < 7 → needs improvement (jump
when FALSE)
; JL is the INVERTED condition: we want >= 7
for good
; so we jump when NOT good (< 7)

```

```

; === Average >= 7: GOOD ===
LEA DX, new1
MOV AH, 9
INT 21H
LEA DX, msgGood
MOV AH, 9
INT 21H
JMP done

needs_improvement:
LEA DX, new1
MOV AH, 9
INT 21H
LEA DX, msgBad
MOV AH, 9
INT 21H

done:
MOV AX, 4C00H
INT 21H
MAIN ENDP
END MAIN

```

### Viva Preparation Q&A:

| #   | Viva Question                                                          | Model Answer                                                                                                                                                                                                                                                                                                                             |
|-----|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q 1 | Why is WHILE used here instead of REPEAT-UNTIL?                        | If the TA enters '0' as the very first input, REPEAT-UNTIL would process it once (adding 0 to sum and incrementing count to 1) before checking the stop condition — giving wrong results (count=1 when 0 students entered). WHILE checks '0' BEFORE processing, so count stays 0. Use WHILE when the body should possibly never execute. |
| Q 2 | How does the WHILE loop pattern differ structurally from REPEAT-UNTIL? | WHILE: check at TOP (CMP+JE at beginning) + JMP at BOTTOM = 2 jumps total. REPEAT-UNTIL: body first + check at BOTTOM (CMP+JNE) = 1 jump total. WHILE may execute 0 times. REPEAT-UNTIL always executes at least once.                                                                                                                   |
| Q 3 | Why MOV AH,0 before DIV BL?                                            | Byte DIV divides AX by BL. If AH has garbage (e.g., 5) and AL=26, then AX=0x051A=1306 decimal, and 1306/4=326 — completely wrong! MOV AH,0 ensures AX = just the sum value in AL. This is the most common DIV mistake.                                                                                                                   |
| Q 4 | Why save the quotient in SI after DIV?                                 | AL holds the quotient after DIV. But INT 21H (called for string output) destroys AX including AL. If we print the average message (AH=9 INT 21H) first, AL gets wiped. SI (Source Index) is not touched by INT 21H or AAM, making it a safe parking spot for the result.                                                                 |
| Q 5 | The branching says JL needs_improvement — not JGE good. Why inverted?  | In IF-THEN-ELSE assembly, the conditional jump skips over the IF body when condition is FALSE. We want >= 7 for GOOD. FALSE of >= 7 is < 7, so JL (jump if less). This 'inverted condition' pattern is universal in assembly IF-THEN-ELSE: jump when condition is FALSE to skip the IF body.                                             |

**Q  
6**

**What happens if count is 0 (TA enters '0' immediately) and DIV executes?**

DIV by 0 causes a hardware interrupt (Divide Error / INT 0) — the program crashes in EMU8086. This is a critical bug. In production code: `CMP DH,0 / JE skip_average` before the DIV to handle zero-count gracefully. Always guard against division by zero.



## Problem 6: The Digital Scoreboard — Combining Everything



TOPICS COVERED: Lab 01: All arithmetic • Lab 02: Full I/O • Lab 03A: IF-THEN-ELSE + CASE • Lab 03B: Nested loops • ALL LABS combined

### Scenario:

You are building the scoreboard software for BRACU's inter-department quiz competition.

The competition has exactly 3 rounds. In each round:

- The host enters a round number (1, 2, or 3) — display the round name  
Round 1 → 'ROUND ONE', Round 2 → 'ROUND TWO', Round 3 → 'ROUND THREE'
- Then enter the score for that round (single digit 1-9)

After all 3 rounds:

- Display total score
- Display performance tier:  
Total 20-27 → 'CHAMPION'  
Total 15-19 → 'RUNNER UP'  
Total 10-14 → 'PARTICIPANT'  
Total < 10 → 'ELIMINATED'

Use: LOOP for 3 rounds, CASE for round names, IF-THEN-ELSE chain for performance tier.

This problem combines EVERY concept from Labs 01-04.



### Your Task:

- LOOP CX=3 for 3 rounds (outer loop)
- Inside loop: read round number → CASE for round name → read score → accumulate
- After loop: compare total with tier thresholds using chained IF-ELSE
- Display total using AAM for 2-digit output
- CASE + LOOP + chained IF-ELSE + I/O + arithmetic ALL in one program

### Complete Solution (with line-by-line comments):

```

.MODEL SMALL
.STACK 100H
.DATA
    askRound DB "Enter round number (1-3): $"
    askScore DB "Enter score (1-9): $"
    rnd1      DB "=== ROUND ONE ===$"
    rnd2      DB "=== ROUND TWO ===$"
    rnd3      DB "=== ROUND THREE ===$"
    msgTotal  DB "TOTAL SCORE: $"
    champion  DB "CHAMPION!$"
    runnerUp  DB "RUNNER UP$"
    particip  DB "PARTICIPANT$"
    elimntd   DB "ELIMINATED$"
    newl      DB 13, 10, "$"

.CODE

```

```

MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, 3                ; CX = 3 rounds (LOOP counter - protect this!)
    MOV DH, 0                ; DH = total score accumulator

ROUND_LOOP:                  ; outer loop: 3 iterations for 3 rounds
    LEA DX, newl
    MOV AH, 9
    INT 21H

    ; === Ask for round number ===
    LEA DX, askRound
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H                  ; round number character → AL
    MOV BL, AL               ; save round char in BL (protects from INT
21H)

    LEA DX, newl             ; newline before round name
    MOV AH, 9
    INT 21H

    ; === CASE: display round name based on BL ===
    CMP BL, '1'
    JE  show_round1
    CMP BL, '2'
    JE  show_round2
    CMP BL, '3'
    JE  show_round3
    JMP get_score            ; unknown round - skip name, still get score

show_round1:
    LEA DX, rnd1             ; 'ROUND ONE'
    MOV AH, 9
    INT 21H
    JMP get_score            ; MUST jump - prevents fall-through to
show_round2!

show_round2:
    LEA DX, rnd2
    MOV AH, 9
    INT 21H
    JMP get_score

show_round3:
    LEA DX, rnd3
    MOV AH, 9
    INT 21H

get_score:                  ; all CASE paths converge here

```

```

LEA DX, new1
MOV AH, 9
INT 21H

; === Ask for score this round ===
LEA DX, askScore
MOV AH, 9
INT 21H
MOV AH, 1
INT 21H          ; score character → AL
SUB AL, 30h       ; ASCII → numeric (e.g. '8' → 8)
MOV AH, 0         ; clear AH before ADD to BX
ADD DH, AL        ; accumulate score into DH
                  ; DH is safe from LOOP (CX) and INT 21H

LOOP ROUND_LOOP  ; CX-- ; jump back if CX != 0
                  ; automatically handles 3 iterations — no
manual counter

; =====
; After 3 rounds: display total and tier
; =====
LEA DX, new1
MOV AH, 9
INT 21H
LEA DX, msgTotal
MOV AH, 9
INT 21H

; Display DH (total) as 2 digits using AAM
MOV AL, DH
AAM              ; AH=tens, AL=units of total score
ADD AH, 30h
ADD AL, 30h
MOV SI, AX       ; save both digits: SIH=tens ASCII, SIL=units
ASCII
                  ; wait — SI is 16-bit: SH=AH, SL=AL... use
BH,BL instead
MOV BH, AH       ; BH = tens ASCII digit
MOV BL, AL       ; BL = units ASCII digit
MOV DL, BH
MOV AH, 2
INT 21H          ; print tens
MOV DL, BL
MOV AH, 2
INT 21H          ; print units

; =====
; Chained IF-ELSE for performance tier
; Check ranges from highest to lowest
; =====
LEA DX, new1
MOV AH, 9
INT 21H

```

```

    CMP DH, 20                ; is total >= 20?
    JGE tier_champion        ; yes → CHAMPION (20-27)

    CMP DH, 15                ; is total >= 15?
    JGE tier_runnerup        ; yes → RUNNER UP (15-19)

    CMP DH, 10                ; is total >= 10?
    JGE tier_participant     ; yes → PARTICIPANT (10-14)

    ; Falls through here if total < 10
    LEA DX, elimntd          ; ELIMINATED
    MOV AH, 9
    INT 21H
    JMP done

tier_champion:
    LEA DX, champion
    MOV AH, 9
    INT 21H
    JMP done

tier_runnerup:
    LEA DX, runnerUp
    MOV AH, 9
    INT 21H
    JMP done

tier_participant:
    LEA DX, particip
    MOV AH, 9
    INT 21H

done:
    MOV AX, 4C00H
    INT 21H
MAIN ENDP
END MAIN

```



### Viva Preparation Q&A:

| #   | Viva Question                                                                           | Model Answer                                                                                                                                                                                                                                                                                                                        |
|-----|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q 1 | <b>This combines CASE inside LOOP. What is the key danger?</b>                          | CX is the LOOP counter. The CASE structure uses JE/JMP — none of these touch CX. But any instruction that modifies CX (like LOOP itself, or accidentally using CL/CH for data) would corrupt the counter. Always check: does anything inside the loop body write to CX, CL, or CH? If yes, restructure to use a different register. |
| Q 2 | <b>Why does the chained IF-ELSE check &gt;= 20 first, then &gt;= 15, then &gt;= 10?</b> | Order matters. If total=22 and we check >= 10 first, it jumps to PARTICIPANT — wrong! By checking the HIGHEST threshold first (>=20), we correctly identify CHAMPION. If >=20 fails, we know total<20, so >=15                                                                                                                      |

|            |                                                                                   |                                                                                                                                                                                                                                                                                                                                                          |
|------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            |                                                                                   | can correctly identify RUNNER UP. This top-down approach works like cascading if-else if in Java.                                                                                                                                                                                                                                                        |
| <b>Q 3</b> | <b>Why is ELIMINATED handled by fall-through instead of a JE check?</b>           | By the time execution reaches the fall-through, we've already verified <code>total&lt;20</code> , <code>total&lt;15</code> , <code>total&lt;10</code> . There's no condition left to check — it MUST be eliminated. Using fall-through (no jump needed) is more efficient. This 'last else' pattern means: everything that didn't match above ends here. |
| <b>Q 4</b> | <b>What if the host enters the same round number twice (e.g., round 1 twice)?</b> | The code doesn't prevent this — it just displays the round name for whatever number is entered. No validation. The loop still runs 3 times regardless. Robust code would track which rounds were entered (e.g., flag variables) and reject duplicates, but that's beyond this scope.                                                                     |
| <b>Q 5</b> | <b>Trace the program for scores 9,8,7 across rounds 1,2,3. What is output?</b>    | Round 1→ROUND ONE, score 9. Round 2→ROUND TWO, score 8. Round 3→ROUND THREE, score 7. Total = 9+8+7 = 24. AAM on 24: AH=2,AL=4 → prints '24'. 24 >= 20 → JGE tier_champion → prints 'CHAMPION!'.                                                                                                                                                         |
| <b>Q 6</b> | <b>Why save AH after AAM before calling INT 21H?</b>                              | AAM puts tens digit in AH. But INT 21H (AH=2 or AH=9) overwrites AH with the function code. If we call INT 21H before saving AH, the tens digit is lost. Solution: MOV BH,AH immediately after AAM to park it safely. Then use MOV DL,BH / MOV AH,2 / INT 21H to print it.                                                                               |